
Introduction and Scope

The goal of this project was to build and evaluate a content-based recommender system by processing raw movie reviews. The scope included loading and cleaning millions of reviews, building two different text-based models (TF-IDF and Binary/Jaccard), and then evaluating their performance, measuring their ability to predict user ratings (using MAE/RMSE) and their effectiveness at Top-N recommendation (using Hit Ratio).

Data Loading and Processing

The goal of this project was to build and evaluate a content-based recommender system by processing raw movie reviews. The scope included building two text-based models (TF-IDF and Binary/Jaccard) and evaluating their performance at predicting user ratings (MAE) and Top-N recommendations (Hit Ratio).

To begin, I loaded the data. I sampled 20% of the user ratings to save memory and loaded all 2.6 million movie reviews. The reviews were then processed: I grouped them all by movie and cleaned the text by removing punctuation, numbers, and stop words.

I saved this cleaned data to a single .parquet file. This was a crucial checkpoint, as I faced several memory crashes and had to restart my notebook. Instead of re-running the 10-minute cleaning process each time, I made a "New Setup" cell that just loaded this saved file, which was much faster.

```
... Mounting Google Drive...
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Loading and sampling /content/drive/My Drive/resources/genome_2021/movie_dataset_public_final/raw/ratings.json...
Loaded 5699763 sample ratings.
Loading /content/drive/My Drive/resources/genome_2021/movie_dataset_public_final/raw/metadata.json...
Loaded metadata.
Loading checkpointed text data from /content/drive/My Drive/resources/processed_for_colab/cleaned_movie_docs.parquet...
Loaded cleaned text data.

--- All Data Loaded Successfully ---
Ratings (Sampled): (5699763, 3)
Cleaned Text: (52081, 2)
Metadata: (84661, 7)
```

As the output shows, this step successfully converted 2.6 million reviews into 52,081 clean documents, one for each movie.

Evaluation Methodology and Baselines

To test the models, I split the ratings into training (80%) and test (20%) sets. I then took a 5,000-rating sample from the test set for all my evaluations. Before building the main models, I created two simple baselines to set a benchmark score:

```
... Splitting ratings_df (shape: (5699763, 3)) into 80/20 train/test sets...
  -> Train set shape: (4559810, 3)
  -> Test set shape: (1139953, 3)

Creating a test sample of 5000 ratings...
  -> test_sample_df shape: (5000, 3)

--- Test Sample Head ---
   item_id  user_id  rating
2610196    99750   965468    3.5
2513317    4034   476438    5.0
4207947    2080   56510    4.0
277215     471   99588    2.0
1188589    1264   912913    3.0
```

Random Baseline: A random guess (MAE: 1.586), which was very inaccurate.

Global Average Baseline: Always guessed the average movie rating (MAE: 0.835), which was much better.

```
... Calculating baselines using 5000 sample ratings...

--- 1. Random Baseline ---
  -> Random MAE: 1.5861
  -> Random RMSE: 1.9565

--- 2. Global Average Baseline ---
Global average rating (from train set): 3.5292
  -> Global Avg MAE: 0.8355
  -> Global Avg RMSE: 1.0545

--- LIVE RESULTS SUMMARY ---
              mae      rmse  hit_ratio
random_baseline 1.58610  1.956489      NaN
avg_baseline    0.83554  1.054528      NaN
```

The goal was to create a model with an MAE lower than 0.835.

Model 1: TF-IDF with Cosine Similarity

My first model used TF-IDF, a method that identifies the *most important* words for each movie. My original plan was to pre-calculate a massive 52,081 x 52,081 similarity matrix, but this repeatedly crashed the notebook.

To solve this, I switched to an "on-the-fly" method. This new function only calculated the similarity between a target movie and the specific movies a user had already rated, which prevented any crashes. This model successfully beat the baseline, achieving an MAE of 0.761.

```

... TF-IDF + On-the-Fly Cosine
Vectorizing cleaned text with TF-IDF...
-> TF-IDF matrix shape: (52081, 20000)
-> Vectorizing finished in 229.65 sec
Creating helper mappings for evaluation...
Grouping training data by user for fast lookup...

TF-IDF and Cosine Model
Running prediction loop for 5000 sample ratings...
-> Prediction loop finished in 85.19 sec
-> Cosine Model MAE: 0.7607
-> Cosine Model RMSE: 0.9855

--- LIVE RESULTS SUMMARY ---

```

	mae	rmse	hit_ratio
random_baseline	1.586100	1.956489	NaN
avg_baseline	0.835540	1.054528	NaN
cosine_tfidf	0.760731	0.985474	NaN

Model 2: Binary Counts with Jaccard Similarity

My second model was for comparison. It used a simpler Binary method and Jaccard Similarity to measure the overlap in words.

Just like the first model, this one also had to use an "on-the-fly" prediction loop. This was necessary because pre-calculating all Jaccard scores for 52,000 movies would have been too slow and would have crashed the notebook. This model proved to be the most accurate, achieving the lowest error score of the entire project with an MAE of 0.759.

```

... Binary Counts + Jaccard Similarity
Vectorizing cleaned text with Binary CountVectorizer...
-> Binary matrix shape: (52081, 20000)
-> Vectorizing finished in 229.95 sec
Converting binary matrix to dense array for loop...
-> Dense vectors shape: (52081, 20000)

Binary and Jaccard Model
Running prediction loop for 5000 sample ratings...
-> Prediction loop finished in 1087.61 sec
-> Jaccard Model MAE: 0.7593
-> Jaccard Model RMSE: 0.9850

--- LIVE RESULTS SUMMARY ---

```

	mae	rmse	hit_ratio
random_baseline	1.586100	1.956489	NaN
avg_baseline	0.835540	1.054528	NaN
cosine_tfidf	0.760731	0.985474	NaN
jaccard_binary	0.759319	0.984968	NaN

Top-N Recommendation & Hit Ratio

This test focused only on movies users "liked" (rated 4.0 or higher). The method involved creating a "user profile" for each user by averaging the content vectors of their liked movies. This profile was then compared to all 52,000+ movies to generate a Top-10 list, and I checked if a "liked" movie from the test set appeared on that list.

This proved to be a major computational challenge. The cosine_tfidf model's Hit Ratio was successfully calculated at 0.2%. This low number is realistic, as it's very difficult for a pure content-based model to find a correct item in a massive catalog. The similar calculation for the jaccard_binary model was stopped after running for over two hours, as it was not computationally feasible to complete.

```
... TF-IDF Hit Ratio (On-the-fly)
Using 2488 'liked' items from the test sample.
Running TF-IDF Hit Ratio loop...
-> TF-IDF loop finished in 1565.68 sec
-> TF-IDF Hits: 5 (out of 2488)
-> TF-IDF Hit Ratio @ 10: 0.0020

TF-IDF Hit Ratio Complete
Cleared temp dataframes from memory.
```

Final Results and Conclusion

The project was a success, as both content-based models clearly outperformed the non-personalized baseline. The final results show that the jaccard_binary model was slightly better at predicting ratings. The cosine_tfidf model's Hit Ratio was 0.2%, which seems low but is realistic for a pure content-based model recommending from a catalog of over 52,000 movies.

The Hit Ratio calculation for the jaccard_binary model was stopped after running for over two hours. This was a necessary trade-off to complete the project and report the other findings.

```
FINAL RESULTS SUMMARY
      mae      rmse hit_ratio
Model
random_baseline 1.586100 1.956489      NaN
avg_baseline    0.835540 1.054528      NaN
cosine_tfidf     0.760731 0.985474    0.00201
jaccard_binary   0.759319 0.984968      NaN
```

```
FINAL RESULTS
| Model | mae | rmse | hit_ratio |
|:-----:|:-----:|:-----:|:-----:|
| random_baseline | 1.5861 | 1.95649 | nan |
| avg_baseline | 0.83554 | 1.05453 | nan |
| cosine_tfidf | 0.760731 | 0.985474 | 0.00200965 |
| jaccard_binary | 0.759319 | 0.984968 | nan |
```

```
Saving final results dictionary to /content/drive/My Drive/resources/processed_for_colab/final_results_summary.json...
Project complete.
```

Overall, this project demonstrated that it's possible to build an effective recommender system from scratch. The main challenges were computational, requiring me to upgrade to a High-RAM machine and develop "on-the-fly" calculations to avoid memory crashes. The final models prove that processing raw text is a valid and successful strategy for building a recommender.